ОТЧЁТ

по лабораторной работе №13

«MongoDB и Redis: документы, кэш и быстрый доступ»

Вариант 20. Интернет-провайдер — тарифы

1. Цель работы

Научиться хранить тариф документом, вести рейтинг популярности, кэшировать сравнение тарифов и расчёт стоимости.

2. Постановка индивидуального задания

Вариант 20. Коллекция tariff содержит название, скорость, цену, ограничения, услуги, регион и статус. В Redis хранятся популярные тарифы, результаты сравнения и кэш расчёта стоимости.

Локальный запуск использует совместимые in-memory серверы с интерфейсом MongoDB и Redis. Операции те же: insert, find, sorted set и ключ с временем жизни.

3. Краткие теоретические сведения

Документная база хранит тариф одним JSON-подобным документом: список услуг не требует отдельной таблицы. Это удобно, когда состав полей может отличаться.

Redis хранит данные в памяти и подходит для счётчиков и коротких результатов. Sorted set упорядочивает тарифы по числу просмотров. SETEX записывает значение и срок, после которого ключ исчезает. Повтор сравнения не обращается к коллекции, пока ключ жив.

4. Описание программы

При старте в коллекцию попадают четыре тарифа, включая архивный. Просмотр увеличивает счётчик в popular_tariffs. Сравнение двух тарифов собирает строку скоростей и цен и кладёт её в Redis. Расчёт стоимости прибавляет 100 рублей за каждую услугу и на повторном вызове помечает источник как cache.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №13. Вариант 20.
Тарифы в документной БД и кэш Redis.
Локальный запуск использует совместимые in-memory серверы с тем же API.
"""

import sys

import fakeredis
import mongomock

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



class TariffApp:
    def __init__(self):
        self.mongo = mongomock.MongoClient()
        self.tariffs = self.mongo["provider"]["tariff"]
        self.redis = fakeredis.FakeRedis(decode_responses=True)
        self._seed()

    def _seed(self):
        if self.tariffs.count_documents({}) == 0:
            self.tariffs.insert_many([
                {"name": "Домашний 100", "speed": 100, "price": 500, "limits": "безлимит",
                 "services": ["антивирус"], "region": "Москва", "status": "активен"},
                {"name": "Домашний 300", "speed": 300, "price": 800, "limits": "безлимит",
                 "services": ["ТВ"], "region": "Казань", "status": "активен"},
                {"name": "Корпоративный 500", "speed": 500, "price": 2500, "limits": "гарантия 99%",
                 "services": ["белый IP", "статический IP"], "region": "Москва", "status": "активен"},
                {"name": "Архив 50", "speed": 50, "price": 200, "limits": "100 ГБ",
                 "services": [], "region": "Тула", "status": "архив"},
            ])

    def list_tariffs(self):
        return list(self.tariffs.find({}, {"_id": 0}))

    def remember_popular(self, name):
        self.redis.zincrby("popular_tariffs", 1, name)

    def popular(self, limit=3):
        return self.redis.zrevrange("popular_tariffs", 0, limit - 1, withscores=True)

    def compare(self, left, right):
        key = f"compare:{left}:{right}"
        cached = self.redis.get(key)
        if cached:
            return cached, "cache"
        a = self.tariffs.find_one({"name": left})
        b = self.tariffs.find_one({"name": right})
        if not a or not b:
            raise ValueError("один из тарифов не найден")
        text = (
            f"{a['name']}: {a['speed']} Мбит/с за {a['price']} руб.; "
            f"{b['name']}: {b['speed']} Мбит/с за {b['price']} руб."
        )
        self.redis.setex(key, 120, text)
        return text, "mongo"
```

```
    def quote(self, name):
        key = f"quote:{name}"
        cached = self.redis.get(key)
        if cached:
            return float(cached), "cache"
        tariff = self.tariffs.find_one({"name": name})
        if tariff is None:
            raise ValueError("тариф не найден")
        extra = 100 * len(tariff.get("services") or [])
        total = tariff["price"] + extra
        self.redis.setex(key, 60, str(total))
        return total, "mongo"


def print_tariff(item):
    services = ", ".join(item["services"]) if item["services"] else "нет"
    print(
        f"{item['name']}: {item['speed']} Мбит/с, {item['price']} руб., "
        f"{item['region']}, {item['status']}, услуги: {services}, лимит: {item['limits']}"
    )


def print_menu():
    print()
    print("===== MongoDB + Redis: тарифы =====")
    print("1. Документы тарифов")
    print("2. Отметить просмотр популярного тарифа")
    print("3. Популярные тарифы")
    print("4. Сравнить два тарифа")
    print("5. Кэш расчёта стоимости")
    print("0. Выход")
    print()


def main():
    app = TariffApp()
    print("Коллекция tariff и ключи Redis готовы.")
    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                for item in app.list_tariffs():
                    print_tariff(item)
            elif choice == "2":
                name = read_line("Название тарифа: ")
                app.remember_popular(name)
                print("Счётчик просмотров увеличен.")
            elif choice == "3":
                rows = app.popular()
                if not rows:
                    print("Популярных тарифов ещё нет.")
                for name, score in rows:
                    print(f"  {name}: {int(score)}")
            elif choice == "4":
                left = read_line("Первый тариф: ")
                right = read_line("Второй тариф: ")
                text, source = app.compare(left, right)
                print(f"[{source}] {text}")
                text2, source2 = app.compare(left, right)
                print(f"Повтор [{source2}] {text2}")
            elif choice == "5":
                name = read_line("Название тарифа: ")
                total, source = app.quote(name)
                print(f"[{source}] к оплате {total:.0f} руб.")
                total2, source2 = app.quote(name)
                print(f"Повтор [{source2}] {total2:.0f} руб.")
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 5.")
        except ValueError as exc:
            print(f"Ошибка: {exc}")
```

```
        except EOFError:
            break


if __name__ == "__main__":
    main()
```

6. Примеры работы программы

Снимки показывают документы тарифов, рейтинг просмотров, сравнение и кэш стоимости.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python13> python main.py

Коллекция tariff и ключи Redis готовы.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 1
Домашний 100: 100 Мбит/с, 500 руб., Москва, активен, услуги: антивирус, лимит: безлимит
Домашний 300: 300 Мбит/с, 800 руб., Казань, активен, услуги: ТВ, лимит: безлимит
Корпоративный 500: 500 Мбит/с, 2500 руб., Москва, активен, услуги: белый IP, статический IP, лимит: гарантия 99%
Архив 50: 50 Мбит/с, 200 руб., Тула, архив, услуги: нет, лимит: 100 ГБ

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 2
Название тарифа: Домашний 100
Счётчик просмотров увеличен.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 2
Название тарифа: Домашний 100
Счётчик просмотров увеличен.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 2
Название тарифа: Корпоративный 500
Счётчик просмотров увеличен.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 3
  Домашний 100: 2
  Корпоративный 500: 1

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 1 – Документы тарифов и рейтинг популярности

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python13> python main.py

Коллекция tariff и ключи Redis готовы.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 4
Первый тариф: Домашний 100
Второй тариф: Корпоративный 500
[mongo] Домашний 100: 100 Мбит/с за 500 руб.; Корпоративный 500: 500 Мбит/с за 2500 руб.
Повтор [cache] Домашний 100: 100 Мбит/с за 500 руб.; Корпоративный 500: 500 Мбит/с за 2500 руб.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 5
Название тарифа: Корпоративный 500
[mongo] к оплате 2700 руб.
Повтор [cache] 2700 руб.

===== MongoDB + Redis: тарифы =====
1. Документы тарифов
2. Отметить просмотр популярного тарифа
3. Популярные тарифы
4. Сравнить два тарифа
5. Кэш расчёта стоимости
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 2 – Сравнение тарифов и кэш расчёта стоимости

7. Ответы на контрольные вопросы

1. Почему тариф здесь документ, а не набор таблиц?

Услуги и ограничения лежат внутри одного описания. Для каталога тарифов так проще читать и отдавать объект целиком.

2. Что хранит sorted set?

Имя тарифа и число просмотров. Выборка с конца даёт самые популярные.

3. Зачем кэшировать сравнение?

Текст сравнения не меняется, пока не изменились сами тарифы. Повторный запрос дешевле чтения двух документов.

4. Что означает TTL?

Время жизни ключа. После него результат считается устаревшим и будет посчитан заново.

5. Чем Redis отличается от MongoDB в этой работе?

MongoDB хранит каталог тарифов. Redis хранит производные и короткие данные: рейтинг, сравнение и сумму.

8. Вывод

В лабораторной работе №13 каталог тарифов варианта 20 хранится документами, а популярность, сравнение и стоимость — в ключах Redis. Повторные операции чтения приходят из кэша, не перечитывая коллекцию.